Gen AI Internship — Week 5

AI for Programming & Development — Short Report

Submitted by: Aniketh | Davine Technologies | 31 August 2026

Application: Expense Tracker (CLI)

I built a small command-line expense tracker in Python — add expenses, view them by category, generate a spending summary, and persist everything to a local JSON file. I picked this over something more trivial (like a to-do list) because it has enough real logic — validation, aggregation, file I/O — to actually need debugging and testing, not just boilerplate.

How I Used AI During Development

Code generation

I described the core class (add/view/summarize/delete expenses, JSON persistence) and asked for a first implementation. I reviewed the generated structure before accepting it — mainly checking that validation happened in one place (add_expense) rather than scattered across the CLI, since that's easier to test and maintain.

Code explanation

I wasn't immediately sure why the AI used a try/except around float(amount) instead of just checking isinstance(amount, (int, float)). I asked it to explain, and the reasoning made sense: CLI input always arrives as a string, so isinstance would reject every valid number typed by a user. Worth knowing before I copied that pattern elsewhere.

Debugging

For the hands-on activity, I deliberately worked from a buggy version of this same app (see Before/After section below) and used AI to help trace three real issues: a syntax error (= instead of == in a comparison), a crash on first run because the file-loading code assumed expenses.json already existed, and a silent logic bug that let negative amounts through and produced a nonsensical negative average. All three are documented with actual terminal output, not just described.

Testing

I asked AI to draft unit tests covering both the happy path and edge cases (empty category, negative amount, non-numeric amount, bad date format, empty-list summary). I added the persistence test (save then reload) myself after noticing the AI-generated set didn't check that data actually survives a save/reload cycle, which felt like an obvious gap for a file-backed app.

What I Changed or Rejected

- Rejected the AI's first suggestion to silently ignore invalid expenses instead of raising an exception — a CLI tool should tell the user why their input was rejected, not fail quietly.
- Added the ExpenseError custom exception myself instead of using bare ValueError everywhere, since it makes the CLI's except blocks clearer about what they're actually catching.
- Simplified the AI's date-validation regex into a straightforward datetime.strptime check — same result, fewer places for an edge case to slip through.

Testing Summary

All 12 unit tests pass (python3 -m unittest test_expense_tracker.py -v). Coverage includes valid input, every validation rejection path, category filtering, summary math including the zero-expenses edge case, deletion (valid and out-of-range index), and save/reload persistence.

Prompts Used

- "Build a Python class for a command-line expense tracker: add expenses with amount/category/description/date, view all or by category, get a summary of totals by category, delete by index, and persist to a JSON file. Include input validation."
- "Why did you use try/except with float(amount) here instead of isinstance checking? Walk me through the reasoning."
- "Here's a Python script with a bug: [buggy code]. It throws a SyntaxError on line 25. What's wrong and how do I fix it?"
- "After fixing that, it now throws a FileNotFoundError on the first run. Why, and what's the correct way to handle a data file that might not exist yet?"
- "This function lets negative amounts through and returns a negative average, which doesn't make sense for an expense tracker. Where should I add validation to prevent that?"
- "Write unit tests for this ExpenseTracker class covering normal use and edge cases — invalid amounts, empty categories, bad dates, and an empty expense list."
- "Write a clear README for this project covering what it does, how to run it, and how to run the tests."